

TAREA_2_UCBM

May 13, 2025

1 Importamos las bibliotecas y cargamos los datos

```
[4]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import math
```

```
[5]: df = pd.read_csv("HW02_data_champ.csv", delimiter = ";", header = 0)
df
```

```
[5]:
```

	PID	CHOSEN	GENDER	AGE	WORK	GRAD	RELIG	STUDENT	MARRIED	PURCH	\
0	1	2	1	44	1	1	4	1	1	2	
1	2	2	1	28	2	1	1	1	1	1	
2	3	2	2	46	2	1	1	2	3	2	
3	4	2	2	24	1	1	1	3	1	1	
4	5	1	2	41	2	1	1	1	1	1	
...	...	...	...	...	...	...	...	...	...	...	
2645	2646	4	2	32	2	3	2	2	3	2	
2646	2647	2	1	34	2	2	1	3	1	1	
2647	2648	3	2	46	1	1	2	1	1	2	
2648	2649	2	1	34	2	1	1	2	1	2	
2649	2650	3	2	39	1	1	1	2	2	2	
...	...	...	...	...	...	...	...	...	...	...	
...	...	...	...	...	...	...	...	...	...	...	
0	...	4	4	4	4	4	4	4	4	4	
1	...	5	2	5	4	5	4	5	4	4	
2	...	5	4	5	4	4	4	4	4	4	
3	...	4	5	3	4	4	4	4	1	1	
4	...	4	4	4	4	5	3	3	3	3	
...	...	...	...	...	...	...	...	...	...	...	
2645	...	4	5	4	4	4	4	4	5	5	
2646	...	4	3	4	4	4	3	3	3	3	
2647	...	5	4	4	5	5	5	3	3	3	
2648	...	4	4	4	5	1	3	3	3	3	

	...	SMELL_HM	COLOR_DEL	COLOR_GOS	COLOR_VCP	COLOR_HM	ADV_DEL	\
0	...	4	4	4	4	4	4	
1	...	5	2	5	4	5	4	
2	...	5	4	5	4	4	4	
3	...	4	5	3	4	4	1	
4	...	4	4	4	5	3	3	
...	...	...	...	...	...	...	...	
2645	...	4	5	4	4	4	5	
2646	...	4	3	4	4	3	3	
2647	...	5	4	4	5	5	3	
2648	...	4	4	4	5	1	3	

2649	...	4	4	5	4	4	3
	ADV_GOS	ADV_VCP	ADV_HM	UNO			
0	3	1	3	1			
1	3	3	5	1			
2	3	3	5	1			
3	4	3	5	1			
4	4	1	5	1			
...	...	...	...	...			
2645	3	4	4	1			
2646	4	3	3	1			
2647	3	3	3	1			
2648	3	3	3	1			
2649	5	3	3	1			

[2650 rows x 53 columns]

```
[6]: # df['PRICE_DEL'] = df['PRICE_DEL'].astype(int)
# df['PRICE_GOS'] = df['PRICE_GOS'].astype(int)
# df['PRICE_VCP'] = df['PRICE_VCP'].astype(int)
# df['PRICE_HM'] = df['PRICE_HM'].astype(int)
```

```
[7]: # Excluir columnas binarias del DataFrame
df_non_binary = df

# 1. Histogramas para ver distribuciones (excluyendo columnas binarias)
df_non_binary.hist(bins=30, figsize=(15, 10))
plt.tight_layout()
plt.show()

# 2. Boxplots organizados de 3 en 3 por fila
numeric_cols = df_non_binary.select_dtypes(include=[np.number]).columns

# Configurar diseño de los subplots para los boxplots
cols_per_row = 3
total = len(numeric_cols)
rows = math.ceil(total / cols_per_row)

# Crear figura con subplots para boxplots
fig, axes = plt.subplots(rows, cols_per_row, figsize=(6 * cols_per_row, 4 *
↪rows))
axes = axes.flatten() # Aplanar para fácil acceso

# Dibujar cada boxplot
for i, col in enumerate(numeric_cols):
    sns.boxplot(x=df_non_binary[col], ax=axes[i])
    axes[i].set_title(f'Boxplot - {col}', fontsize=10)
```

```

# Ocultar subplots vacíos (si los hay)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

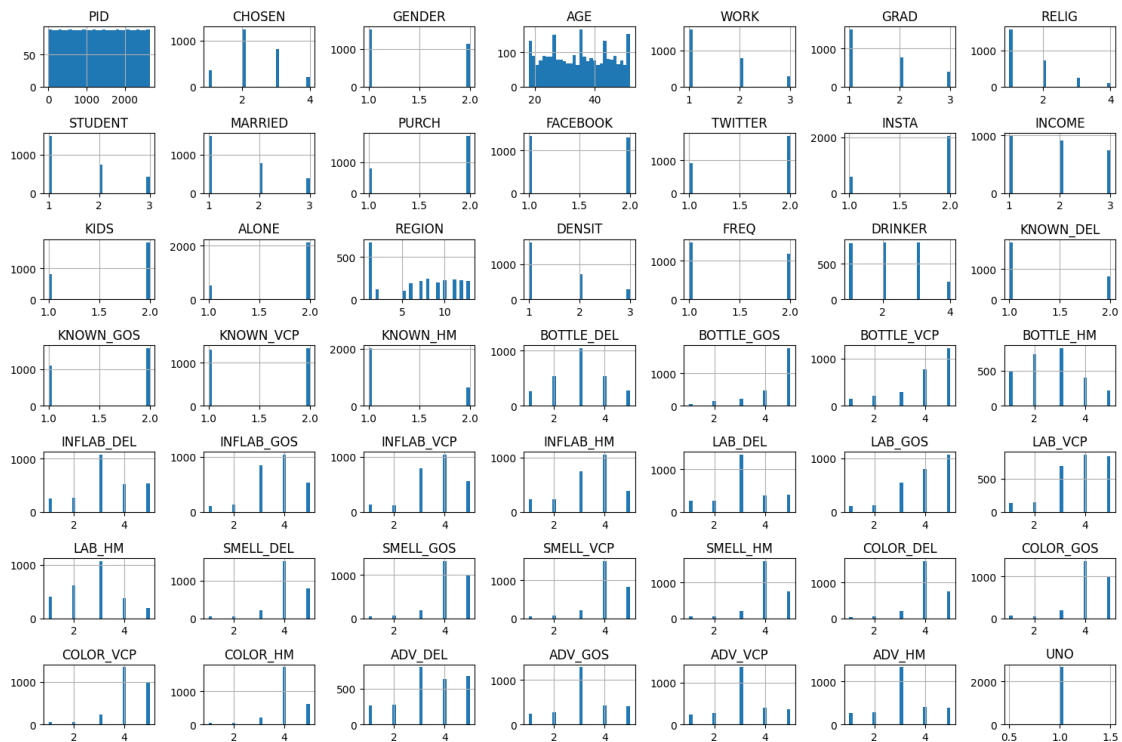
# 3. Violin plots organizados de 3 en 3 por fila
# Crear figura con subplots para los violin plots
fig, axes = plt.subplots(rows, cols_per_row, figsize=(6 * cols_per_row, 4 *
    ↳rows))
axes = axes.flatten() # Aplanar para fácil acceso

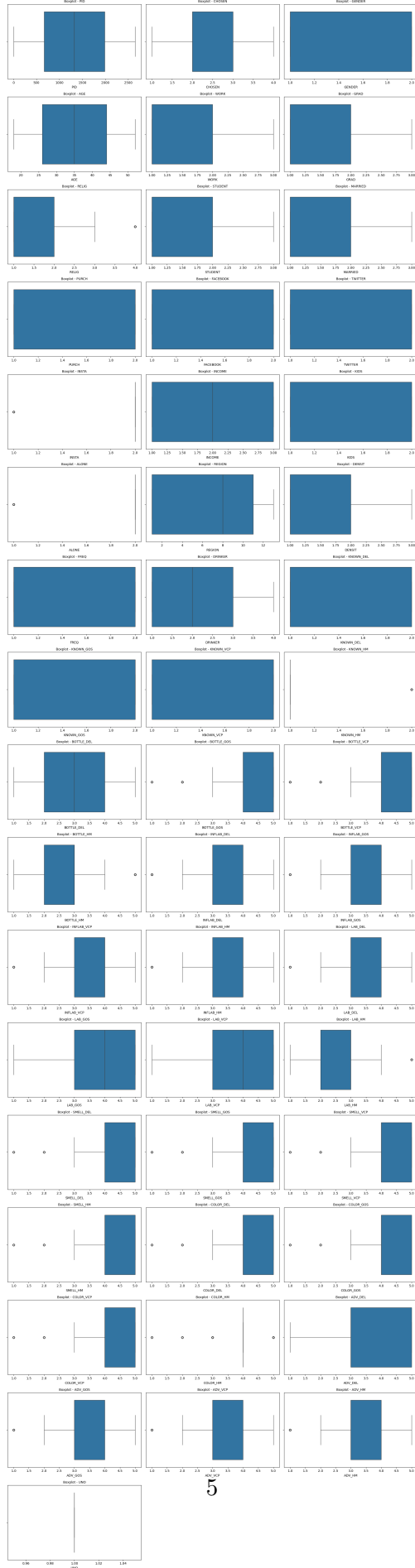
# Dibujar cada violin plot
for i, col in enumerate(numeric_cols):
    sns.violinplot(x=df_non_binary[col], ax=axes[i])
    axes[i].set_title(f'Violin Plot - {col}', fontsize=10)

# Ocultar subplots vacíos (si los hay)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

```






```
[8]: # Calificaciones de los atributos para todas las marcas
columns_to_check = ['BOTTLE_DEL', 'INFLAB_DEL', 'LAB_DEL', 'SMELL_DEL',
                    'COLOR_DEL', 'ADV_DEL',
                    'BOTTLE_GOS', 'INFLAB_GOS', 'LAB_GOS', 'SMELL_GOS',
                    'COLOR_GOS', 'ADV_GOS',
                    'BOTTLE_VCP', 'INFLAB_VCP', 'LAB_VCP', 'SMELL_VCP',
                    'COLOR_VCP', 'ADV_VCP',
                    'BOTTLE_HM', 'INFLAB_HM', 'LAB_HM', 'SMELL_HM', 'COLOR_HM',
                    'ADV_HM']

# Calcular estadísticas descriptivas de las calificaciones para todas las marcas
desc_stats = df[columns_to_check].describe()

# Mostrar estadísticas descriptivas
print("Estadísticas descriptivas de las calificaciones de todas las marcas:")
print(desc_stats)

# Precios de todas las marcas
df_prices = df[['PRICE_DEL', 'PRICE_GOS', 'PRICE_VCP', 'PRICE_HM']]

# Mostrar estadísticas de precios
print("\nEstadísticas de los precios de todas las marcas:")
print(df_prices.describe())
```

Estadísticas descriptivas de las calificaciones de todas las marcas:

	BOTTLE_DEL	INFLAB_DEL	LAB_DEL	SMELL_DEL	COLOR_DEL	\
count	2650.000000	2650.000000	2650.000000	2650.000000	2650.000000	
mean	3.018491	3.316604	3.152830	4.123774	4.127925	
std	1.103222	1.178223	1.108515	0.792900	0.742811	
min	1.000000	1.000000	1.000000	1.000000	1.000000	
25%	2.000000	3.000000	3.000000	4.000000	4.000000	
50%	3.000000	3.000000	3.000000	4.000000	4.000000	
75%	4.000000	4.000000	4.000000	5.000000	5.000000	
max	5.000000	5.000000	5.000000	5.000000	5.000000	

	ADV_DEL	BOTTLE_GOS	INFLAB_GOS	LAB_GOS	SMELL_GOS	...	\
count	2650.000000	2650.000000	2650.000000	2650.000000	2650.000000	...	
mean	3.445283	4.425283	3.647547	3.980755	4.203774	...	
std	1.249321	0.968688	0.993145	1.080738	0.829788	...	
min	1.000000	1.000000	1.000000	1.000000	1.000000	...	
25%	3.000000	4.000000	3.000000	3.000000	4.000000	...	
50%	3.000000	5.000000	4.000000	4.000000	4.000000	...	
75%	5.000000	5.000000	4.000000	5.000000	5.000000	...	
max	5.000000	5.000000	5.000000	5.000000	5.000000	...	

	LAB_VCP	SMELL_VCP	COLOR_VCP	ADV_VCP	BOTTLE_HM \
count	2650.000000	2650.000000	2650.000000	2650.000000	2650.000000
mean	3.800000	4.132830	4.167547	3.156226	2.676226
std	1.092201	0.794765	0.846820	1.071847	1.175149
min	1.000000	1.000000	1.000000	1.000000	1.000000
25%	3.000000	4.000000	4.000000	3.000000	2.000000
50%	4.000000	4.000000	4.000000	3.000000	3.000000
75%	5.000000	5.000000	5.000000	4.000000	3.000000
max	5.000000	5.000000	5.000000	5.000000	5.000000

	INFLAB_HM	LAB_HM	SMELL_HM	COLOR_HM	ADV_HM
count	2650.000000	2650.000000	2650.000000	2650.000000	2650.000000
mean	3.418113	2.747170	4.113962	4.060755	3.139245
std	1.116933	1.092996	0.761866	0.738117	1.098683
min	1.000000	1.000000	1.000000	1.000000	1.000000
25%	3.000000	2.000000	4.000000	4.000000	3.000000
50%	4.000000	3.000000	4.000000	4.000000	3.000000
75%	4.000000	3.000000	5.000000	4.000000	4.000000
max	5.000000	5.000000	5.000000	5.000000	5.000000

[8 rows x 24 columns]

Estadísticas de los precios de todas las marcas:

	PRICE_DEL	PRICE_GOS	PRICE_VCP	PRICE_HM
count	2650	2650	2650	2650
unique	1	1	1	1
top	15,85	27,35	20,88	12,5
freq	2650	2650	2650	2650

```
[9]: # Crear una figura con subgráficos
fig, axes = plt.subplots(2, 4, figsize=(18, 12)) # 2 filas y 4 columnas
axes = axes.flatten() # Aplanar para fácil acceso

# Violin plots para la marca DEL
sns.violinplot(x='CHOSEN', y='BOTTLE_DEL', data=df[df['CHOSEN'] == 1],
               ax=axes[0])
axes[0].set_title('Violin Plot - Calificación del diseño de la botella DEL')
axes[0].set_xlabel('Marca DEL')
axes[0].set_ylabel('Calificación del diseño de la botella')

sns.violinplot(x='CHOSEN', y='SMELL_DEL', data=df[df['CHOSEN'] == 1],
               ax=axes[1])
axes[1].set_title('Violin Plot - Calificación del aroma de la botella DEL')
axes[1].set_xlabel('Marca DEL')
axes[1].set_ylabel('Calificación del aroma')

# Violin plots para la marca GOS
```

```

sns.violinplot(x='CHOSEN', y='BOTTLE_GOS', data=df[df['CHOSEN'] == 2],
    ↪ax=axes[2])
axes[2].set_title('Violin Plot - Calificación del diseño de la botella GOS')
axes[2].set_xlabel('Marca GOS')
axes[2].set_ylabel('Calificación del diseño de la botella')

sns.violinplot(x='CHOSEN', y='SMELL_GOS', data=df[df['CHOSEN'] == 2],
    ↪ax=axes[3])
axes[3].set_title('Violin Plot - Calificación del aroma de la botella GOS')
axes[3].set_xlabel('Marca GOS')
axes[3].set_ylabel('Calificación del aroma')

# Violin plots para la marca VCP
sns.violinplot(x='CHOSEN', y='BOTTLE_VCP', data=df[df['CHOSEN'] == 3],
    ↪ax=axes[4])
axes[4].set_title('Violin Plot - Calificación del diseño de la botella VCP')
axes[4].set_xlabel('Marca VCP')
axes[4].set_ylabel('Calificación del diseño de la botella')

sns.violinplot(x='CHOSEN', y='SMELL_VCP', data=df[df['CHOSEN'] == 3],
    ↪ax=axes[5])
axes[5].set_title('Violin Plot - Calificación del aroma de la botella VCP')
axes[5].set_xlabel('Marca VCP')
axes[5].set_ylabel('Calificación del aroma')

# Violin plots para la marca HM
sns.violinplot(x='CHOSEN', y='BOTTLE_HM', data=df[df['CHOSEN'] == 4],
    ↪ax=axes[6])
axes[6].set_title('Violin Plot - Calificación del diseño de la botella HM')
axes[6].set_xlabel('Marca HM')
axes[6].set_ylabel('Calificación del diseño de la botella')

sns.violinplot(x='CHOSEN', y='SMELL_HM', data=df[df['CHOSEN'] == 4], ax=axes[7])
axes[7].set_title('Violin Plot - Calificación del aroma de la botella HM')
axes[7].set_xlabel('Marca HM')
axes[7].set_ylabel('Calificación del aroma')

# Ajustar el diseño para que no se solapen los gráficos
plt.tight_layout()
plt.show()

```

